BeyondCorp Enterprise — Zero-Trust Access for Apps & Google Cloud

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — BeyondCorp Enterprise: Zero-Trust Access for Apps & Google Cloud 4

The outcome . . . 4

The approach . . . 4

Why it is safe and governed . . . 4

What is delivered . . . 4

1. Solution Overview — BeyondCorp Enterprise: Zero-Trust Access for Apps & Google Cloud 5

What this solution is . . . 5

How this differs from Workspace Context-Aware Access . . . 5

The reference design . . . 6

Benefits . . . 6

Limitations and boundaries . . . 7

Deployment model . . . 7

2. Requirements — BeyondCorp Enterprise: Zero-Trust Access 7

2.1 Access Context Manager enabled with an access policy on the Organization . . . 7

2.2 A pilot web application behind a Google Cloud HTTPS load balancer . . . 8

2.3 A pilot VM for IAP TCP forwarding (SSH) . . . 8

2.4 A device-trust signal source (and its licensing) . . . 9

2.5 Pilot identity group(s) and least-privilege admin roles . . . 9

2.6 IAP and required APIs enabled on target projects . . . 9

2.7 A documented rollback and change window . . . 10

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace . . . 10

2. Organization-level administrative access . . . 11

3. A Cloud Billing account . . . 11

4. The gcloud / IaC toolchain and API enablement . . . 11

5. A resource-hierarchy, naming & environment design . . . 12

6. Security, network & governance stakeholders and policy . . . 12

3. Implementation — BeyondCorp Enterprise: Zero-Trust Access 12

Stage 0 — Assess (read-only) . . . 13

Stage 1 — Design access levels . . . 13

Stage 2 — Configure IAP for the pilot web app . . . 14

Stage 3 — Configure IAP TCP forwarding for the pilot VM . . . 14

Stage 4 — Pilot & Validate (the deny/allow gate) . . . 15

Stage 5 — Retire the VPN path (production-affecting — change window) . . . 15

Stage 6 — Expand . . . 15

Rollback (keep ready before any enforcement change) . . . 16
5. Testing & Validation — BeyondCorp Enterprise: Zero-Trust Access 16
T1 — Access levels exist and are well-formed (Gate 1) . . . 16
T2 — Allowed path: corp-managed device, allowed region (Gate 4) . . . 17
T3 — Denied path: unmanaged device or disallowed region (Gate 4) . . . 17
T4 — Denied path: unauthorized identity (Gate 4) . . . 17
T5 — Direct-bypass attempt is refused (Gate 2/3/5) . . . 17
T6 — Device-trust signal is actually evaluated . . . 18
T7 — Guardrail / posture check is clean (Gate 4) . . . 18
T8 — VPN retirement verified (Gate 5) . . . 18
6. Troubleshooting — BeyondCorp Enterprise: Zero-Trust Access 18
Access levels won't create / "no access policy" . . . 19
IAP page won't enable for the web app / OAuth consent error . . . 19
Access is granted from an unmanaged device or wrong region (T3 fails) . . . 19
Device state changes but the decision doesn't (T6 fails) . . . 19
A user who should have access is denied (T2 fails) . . . 20
SSH via `--tunnel-through-iap` fails . . . 20
Direct access to the backend/VM still works (T5 fails) . . . 20
An unauthorized user reaches the resource (T4 fails) . . . 20
VPN retirement locked users out (Stage 5) . . . 21
`gcloud` access-level / IAP flags differ from the guide . . . 21
Guardrail check can't read policies/logs . . . 21
7. Best Practices — BeyondCorp Enterprise: Zero-Trust Access 21
Policy design . . . 22
Rollout discipline . . . 22
Keep the two products distinct . . . 22
Device signals & licensing . . . 22
Operations, monitoring & evidence . . . 23
Least privilege for administrators . . . 23
Governance & scope . . . 23
8. Deliverables Summary — BeyondCorp Enterprise: Zero-Trust Access 23
Documentation . . . 23
The 5 Python tools (in `scripts/`) . . . 24
Appendix A — PowerShell Scripts 26

Executive Summary — BeyondCorp Enterprise: Zero-Trust Access for Apps & Google Cloud

The outcome

Access to the organization's internal applications, virtual machines, and Google Cloud resources is granted **per request, on who and what — not where.** A device being inside the VPN no longer makes it trusted; a laptop that fails device posture is denied even from a corporate IP; and remote access to internal apps no longer depends on a VPN concentrator. The end state is **true zero trust**: apps and VMs sit behind Identity-Aware Proxy, context-aware access levels are enforced on every request, and the VPN trust that these apps relied on is retired.

The approach

The solution implements **BeyondCorp Enterprise** — Google Cloud's zero-trust access model — using three native controls working together:

- **Access Context Manager** access levels define reusable **identity + device + context** conditions

(corporate-managed device, allowed region, trusted network).

- **Identity-Aware Proxy (IAP)** becomes the single front door in front of each web app (behind a Google

load balancer) and each VM (via IAP TCP forwarding for SSH), so no request reaches a backend unauthenticated.

- **Context-aware access bindings** attach an access level to each grant, so access is allowed only when

identity **and** context both pass.

This is the **Google Cloud / apps-and-VMs** side of zero trust — distinct from, but complementary to, Workspace Context-Aware Access, which protects Google Workspace apps in the Admin console. The solution is explicit about which console each control belongs in.

Why it is safe and governed

Delivery follows a **pilot-first** model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — against a named pilot app, a named pilot VM, and a named pilot group. Explicit **pass/fail gates** require proof that access is *granted* on identity+context and *denied* for unmanaged devices, wrong regions, unauthorized identities, and direct bypass attempts **before** any VPN path is retired. A **guardrail check** blocks weak configuration (IAP bindings without a context condition, over-broad principals, SSH not locked to the IAP range), every production-affecting change has a **documented rollback**, and IAP and Cloud Audit Logs provide the allow/deny **evidence**.

What is delivered

A complete, self-guided implementation: solution overview, requirements, step-by-step implementation, testing/validation, troubleshooting, and best-practice guides — plus **five safe-by-default Python tools** (assessment, IAP enablement helper, deny/allow validation report, policy guardrail check, and a

scope/config helper) built on `gcloud` and the Google Cloud Client Libraries, with **no hardcoded credentials, keys, or secrets**. Version- and preview-dependent capabilities are cited or flagged "verify in your organization," never invented. The result is enterprise zero-trust access, deployed at a fraction of typical consulting cost.

1. Solution Overview — BeyondCorp Enterprise: Zero-Trust Access for Apps & Google Cloud

What this solution is

This is the repeatable way to replace network-location trust with identity- and context-aware zero-trust access for an enterprise's web applications, virtual machines, and Google Cloud resources. It implements **BeyondCorp Enterprise** — Google Cloud's zero-trust access model — using three building blocks that work together:

- **Access Context Manager** — where you define **access levels**: named, reusable conditions that combine who the caller is with the context of the request (device state, IP/geo, corporate-managed status, time). An access level is a boolean the platform re-evaluates on every request.
- **Identity-Aware Proxy (IAP)** — the enforcement point that sits *in front of* an application or VM. A request never reaches the backend until IAP has authenticated the user and checked that they are both authorized (IAM) and satisfy any bound access level (context). IAP works for HTTPS web apps behind a Google load balancer and, via **IAP TCP forwarding**, for SSH/RDP to VMs — removing the need for a bastion host reachable over a VPN.
- **Context-aware access bindings** — the IAM binding that attaches an access level to a principal's access to a resource, so a grant reads "this group may reach this app **only when** they meet this access level," not just "this group may reach this app."

The result: access is granted **per request, on who and what — not where**. A device inside a VPN is no longer automatically trusted; a laptop that fails device posture is denied even from the corporate IP range; and remote access to internal apps no longer depends on a VPN concentrator at all.

Google Cloud Console layouts, `gcloud` command surfaces, and specific BeyondCorp Enterprise capabilities change over time and are owned by Google. Where a capability, tier, or preview feature is version-dependent (for example, device-trust signal sources, Chrome Enterprise Premium licensing for certain device signals, or IAP support for a given resource type), this solution **cites it or flags "verify in your organization"** rather than inventing an exact path.

How this differs from Workspace Context-Aware Access

The organization may already run **Context-Aware Access (CAA)** in the **Google Admin console** to gate access to **Google Workspace** apps (Gmail, Drive, Calendar) by device and context. That is a **Workspace** control, configured under Admin console → Security → Access and data control → Context-Aware Access, and it protects Google's own SaaS apps.

This solution is the Google Cloud / BeyondCorp Enterprise side of the same idea — it protects **your own applications and VMs and Google Cloud resources**, configured with **Access Context Manager +**

IAP + Cloud IAM in the **Google Cloud Console** (not the Admin console). The two share the same conceptual model (identity + device + context) and can share device-trust signals, but they are distinct products with distinct admin surfaces. This solution is explicit about which console each step belongs in so the two are not conflated. Where Workspace CAA and BeyondCorp access levels can draw on the same device signals, that overlap is noted; the configuration itself is done per surface.

The reference design

The pilot-first reference design has five layers:

1. **Access levels (Access Context Manager)** — a small, named set of reusable conditions, e.g.

`Access Level – Corp-Managed – v1` (corporate-managed device + allowed region) and `Access Level – Trusted-Network – v1` (allowed corporate IP ranges). Basic access levels use attribute conditions; custom access levels use **Common Expression Language (CEL)**. Access levels live at the Organization node so every project can reference them.

1. **Device-trust signals** — the inputs an access level tests. Depending on licensing and enrollment,

signals can come from **endpoint verification / Chrome Enterprise**, corporate-managed device status, or an integrated endpoint-management partner. Because signal sources and their licensing are version- and tier-dependent, the solution names the signal it relies on and flags **"verify the signal source and license in your organization."**

1. **Identity-Aware Proxy (IAP)** — enabled in front of a pilot web app (behind an external/internal

HTTPS load balancer) and, separately, for **IAP TCP forwarding** to a pilot VM for SSH. IAP becomes the single front door; the backend is not reachable directly.

1. **Context-aware access bindings** — IAM policy on the IAP-protected resource that grants the pilot

group `roles/iap.httpsResourceAccessor` (web) or the TCP-forwarding role (VMs) **conditioned on** the access level. Access is allowed only when identity **and** context both pass.

1. **VPN retirement path** — once the app/VM are reachable through IAP with context enforced, the

corresponding VPN route and firewall exposure for that app are removed, so the app is no longer reachable by network position at all.

Benefits

• **No implicit network trust.** A request is judged on identity + device + context every time, so a

compromised device or a caller on the "right" network is not automatically allowed.

• **VPN reduction/retirement.** Internal apps and VM SSH move behind IAP, removing whole classes of

VPN exposure and the flat lateral access a VPN grants.

• **Least-privilege, per-app.** Each app/VM gets its own IAM + access-level policy; there is no

all-or-nothing tunnel.

• **Auditable.** IAP and Cloud Audit Logs record who reached what, from what context — every allow and

deny is evidence.

• **Reusable policy.** Access levels are defined once and referenced across many resources.

Limitations and boundaries

- **IAP resource support is specific.** IAP fronts HTTPS load-balanced web apps, App Engine, and (via TCP forwarding) VMs; not every workload type is IAP-eligible. Confirm eligibility for each target — **verify in your organization.**
- **Device signals depend on licensing/enrollment.** Richer device-trust signals may require Chrome Enterprise Premium / endpoint verification enrollment. The solution states the dependency and flags it for verification rather than assuming a tier.
- **This is not the Workspace CAA product** (see above) and does not configure Workspace app access.
- **Legal/records/compliance determinations are deferred** to the org's legal, records, and compliance functions; this solution provides the technical access-control method and the evidence.
- **IaC is at the pattern level** — Terraform / Cloud Foundation Toolkit blueprints are shown as a pattern with representative snippets; module internals are deferred to the tool's own docs.

Deployment model

This solution follows the house model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first. Access levels and IAP are configured against a **named pilot app and a named pilot VM** with a **named pilot group** first; explicit **pass/fail gates** confirm that access is *granted* on identity+context and *denied* otherwise before any VPN path is retired or the pattern is expanded. Every org-wide or production-affecting change has a **documented rollback**.

2. Requirements — BeyondCorp Enterprise: Zero-Trust Access

This solution assumes the **six shared GCP foundation requirements** in `global-requirements-gcp.md` are already met: a **GCP Organization** linked to Cloud Identity / Workspace, **Organization-level administrative access**, a **Cloud Billing account**, the `gcloud`**/IaC toolchain with API enablement**, a **resource-hierarchy/naming/pilot design**, and **security/network/governance stakeholder engagement**. Do not re-do those here — verify them, then satisfy the solution-specific prerequisites below before starting Implementation.

Follow the naming convention `<Purpose> – <Scope> – <Stage>` for every artifact (e.g. `Access Level – Corp-Managed – v1`, `IAP App – Pilot`, `Pilot Group – ZeroTrust`).

2.1 Access Context Manager enabled with an access policy on the Organization

- **Description:** The **Access Context Manager** API enabled and an **organization-scoped access policy** in place. Access levels are children of this access policy and are referenced by both VPC Service Controls and IAP context-aware access.

• **Why required:** Access levels (the identity+device+context conditions this solution enforces) can

only be created inside an access policy; without it there is nowhere to define them.

• **How to verify:** `gcloud access-context-manager policies list --organization <ORG_ID>` returns a

policy. Confirm the Access Context Manager API is enabled (`gcloud services list --enabled | grep accesscontextmanager`).

• **How to implement if missing:** Enable the API

(`gcloud services enable accesscontextmanager.googleapis.com`) and create the org access policy (`gcloud access-context-manager policies create --organization <ORG_ID> --title "<title>"`). Only one scoped access policy per organization applies to the whole org — **verify** whether one already exists (often created by the Landing Zone or VPC-SC solution) before creating a second.

2.2 A pilot web application behind a Google Cloud HTTPS load balancer

• **Description:** A **named pilot web app** (`IAP App – Pilot`) served through an **external or

internal HTTPS load balancer** with a backend service, on a project you control. IAP for web apps is enabled on the load balancer's backend service.

• **Why required:** IAP for HTTPS protects apps that sit behind a Google load balancer backend service;

the app must be in that shape before IAP can front it.

• **How to verify:** In the Console (Network services → Load balancing) confirm the HTTPS load balancer

and backend service; the app is reachable at its URL over HTTPS.

• **How to implement if missing:** Deploy the pilot app behind an HTTPS load balancer (or App Engine,

which IAP also supports). Exact load-balancer setup is deferred to Google's load-balancing docs; IAP eligibility of the specific app type should be confirmed — **verify in your organization.**

2.3 A pilot VM for IAP TCP forwarding (SSH)

• **Description:** A **named pilot Compute Engine VM** (`IAP VM – Pilot`) that today is reached over SSH

(often via a bastion/VPN), which you will move to **IAP TCP forwarding**.

• **Why required:** IAP TCP forwarding is the mechanism that replaces bastion/VPN SSH access with

context-checked, brokered connections; a target VM is needed to pilot it.

• **How to verify:** The VM exists (`gcloud compute instances list`) and you can currently SSH to it.

• **How to implement if missing:** Create a VM in the pilot project. Ensure a firewall rule allows

ingress on the SSH port **from the IAP TCP forwarding IP range** `35.235.240.0/20` (this is the documented IAP forwarding range) rather than from the internet or the VPN.

2.4 A device-trust signal source (and its licensing)

• **Description:** A source of **device-trust signals** the access levels will test — for example **endpoint verification / Chrome Enterprise** device attributes, corporate-managed device status, or an integrated endpoint-management partner signal.

• **Why required:** The "what" in "who and what" — the device condition in an access level — needs a real signal to evaluate; without it access levels can only test identity and network attributes.

• **How to verify:** Confirm which signal source is available and enrolled for the pilot devices, and confirm the **license tier** that unlocks it (some device signals require **Chrome Enterprise Premium** / endpoint-verification enrollment). **Verify the exact signal source and license in your organization** — these are tier- and version-dependent and must not be assumed.

• **How to implement if missing:** Enroll pilot devices in the chosen signal source and provision the required license before building device-conditioned access levels. If no device signal is yet available, start with an identity + network (IP/geo) access level and add the device condition once the signal source is live.

2.5 Pilot identity group(s) and least-privilege admin roles

• **Description:** A **named pilot group** (`Pilot Group – ZeroTrust`) in Cloud Identity for the users who will test access, and the least-privilege admin roles needed to configure this solution: `roles/accesscontextmanager.policyAdmin` (manage access levels), `roles/iap.admin` (manage IAP settings), and the ability to set IAM on the target resources.

• **Why required:** Bindings target groups, not individuals; and configuring access levels + IAP requires the specific admin roles, granted narrowly rather than via broad org-admin.

• **How to verify:** The group exists and the operator holds the roles (`gcloud organizations get-iam-policy <ORG_ID>` / project-level `get-iam-policy`).

• **How to implement if missing:** Create the pilot group; have a super-admin grant the named roles under change control.

2.6 IAP and required APIs enabled on target projects

• **Description:** The **Identity-Aware Proxy API** (`iap.googleapis.com`) and Compute Engine API enabled on the projects hosting the pilot app and VM, plus the OAuth consent/brand configured for IAP where required for web apps.

• **Why required:** IAP cannot be turned on for a resource until its API is enabled; web IAP requires an OAuth brand/consent screen on the project.

• **How to verify:** `gcloud services list --enabled` shows `iap.googleapis.com`; in the Console the IAP page loads for the project.

- **How to implement if missing:** `gcloud services enable iap.googleapis.com compute.googleapis.com`;

configure the OAuth brand/consent as prompted the first time IAP for web is enabled (steps are version-dependent — follow the in-console prompt / current IAP docs).

2.7 A documented rollback and change window

- **Description:** An agreed **rollback procedure** and **change window** for enabling IAP / binding

access levels / retiring a VPN path, since these change how users reach an app.

- **Why required:** Zero-trust enforcement can lock users out if a signal or binding is wrong; a

rollback (disable IAP enforcement / detach the access-level condition / re-open the prior path) must be ready before production-affecting changes.

- **How to verify:** The rollback steps and approver are written down and the pilot group is briefed.
- **How to implement if missing:** Capture the rollback (see Implementation §rollback) and confirm the

approver before the first enforcement change.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The **Google Cloud Console** (console.cloud.google.com), the* `gcloud` ***CLI**, and **Cloud Identity / Google Admin** are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability, API, or default is **version- or preview-dependent** (new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is **cited or flagged "verify in your organization"** — never invented. **Infrastructure-as-Code** (Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the **pattern level** — the solutions give the design, the* `gcloud`*/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as **Python** (Cloud Client Libraries /* `gcloud` *wrappers), never with hardcoded credentials or keys.*

1. A Google Cloud Organization linked to Cloud Identity / Workspace

- **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists

once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

- **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the

Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm an Organization resource exists and its ID.

• **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

• **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.

• **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.

• **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the specific roles each solution lists) via `gcloud organizations get-iam-policy`.

• **How to implement if missing:** Have a super-admin grant the roles under change control; keep break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

• **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the solution provisions, with a billing administrator available.

• **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing; FinOps/budget work operates directly on the billing account.

• **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.

• **How to implement if missing:** Create/associate a billing account (Google or reseller) before provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

• **Description:** A workstation or pipeline with the **Google Cloud SDK (`gcloud`, `gsutil`)**, **Python 3.10+** with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.

• **Why required:** Configuration, automation, and the provided Python tools run through these; most services require their API to be enabled first.

- **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a

target project.

- **How to implement if missing:** Install the SDK + libraries; enable the named APIs

(`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

- **Description:** An agreed **folder/project structure** (e.g. by department/environment), a

naming/labeling convention, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

- **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and

labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

- **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with

the platform owners.

- **How to implement if missing:** Define the hierarchy and naming standard (this is often the first

solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

- **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps**

stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

- **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to

enterprise security and financial policy, not just technical defaults.

- **How to verify:** Confirm the applicable policies and named approvers/owners for security,

networking, and cost.

- **How to implement if missing:** Identify the owners and capture the policy requirements before

applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials, exported service-account keys committed to disk, or secrets in code.*

3. Implementation — BeyondCorp Enterprise: Zero-Trust Access

Follow the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first. Name every artifact with `<Purpose> – <Scope> – <Stage>` and reuse the exact name in later steps, testing, and the scripts. Commands below use the **Google Cloud Console** and the `gcloud` **CLI**; where a capability is version- or preview-dependent it is flagged **verify** — confirm the current path in your own organization rather than assuming it. The **provided Python tools** (see §8) automate the read-only assessment, dry-run configuration, validation, and guardrail checks referenced here.

Set shell variables once (adapt values):

```
ORG_ID=<your-org-id>
POLICY_ID=<access-context-manager-policy-id>   # from requirement 2.1
PILOT_PROJECT=<pilot-project-id>
PILOT_GROUP=group:zero-trust-pilot@<domain>    # Pilot Group – ZeroTrust
PILOT_VM=iap-vm-pilot                           # IAP VM – Pilot
```

Stage 0 — Assess (read-only)

Goal: know the current access exposure before changing anything.

1. Inventory the current front doors for the pilot app and VM: how are they reached today (public IP,

VPN route, bastion), and what firewall rules expose them.

1. Run the assessment tool (read-only) to capture the baseline:

`python bce_access_assessment.py --project $PILOT_PROJECT --org $ORG_ID`. It reports existing access policies/levels, whether IAP is enabled on candidate resources, and firewall rules that expose SSH or HTTP(S) broadly (a VPN/flat-trust signal). **Gate 0 (record-only):** you have a written baseline of how the pilot app and VM are reachable today.

Stage 1 — Design access levels

Goal: define the reusable identity+device+context conditions.

1. Decide the first two access levels:

• `Access Level – Trusted-Network – v1` — allowed corporate IP CIDR ranges (a basic access level).

• `Access Level – Corp-Managed – v1` — corporate-managed device + allowed region, drawing on the

device-trust signal from requirement 2.4. **Verify** which device attributes your signal source exposes before finalizing this level.

1. Author them as a **dry-run** first with the scope/config helper's build mode, which prints the exact

`gcloud`/YAML it would apply without changing anything: `python bce_scope_config_helper.py build --policy $POLICY_ID --name "Access Level – Trusted-Network – v1" --ip-ranges 203.0.113.0/24 --dry-run`

1. Review the printed definition. Basic access levels use attribute conditions; if you need combined

logic (identity AND device AND region), use a **custom** access level with a **CEL** expression — the helper emits the CEL skeleton to review. Keep expressions in version control.

1. Apply when reviewed (the builder requires explicit `--apply`, or apply the printed `gcloud`

yourself):

```
gcloud access-context-manager levels create trusted_network_v1 \
  --policy $POLICY_ID \
  --title "Access Level – Trusted-Network – v1" \
  --basic-level-spec level.yaml   # the YAML the builder printed
```

Gate 1: both access levels exist and list under `gcloud access-context-manager levels list --policy $POLICY_ID`.

Stage 2 — Configure IAP for the pilot web app

Goal: put IAP in front of `IAP App – Pilot` so no request reaches the backend unauthenticated.

1. Enable IAP for the app's HTTPS load-balancer backend service (Console: Security → Identity-Aware

Proxy → toggle the backend service on; configure the OAuth consent/brand if prompted — this first-run step is version-dependent, **verify** against current IAP docs). The IAP helper tool prints the exact enable steps and checks prerequisites in dry-run: `python bce_iap_enable_helper.py --project $PILOT_PROJECT --resource-type web --backend-service iap-app-pilot-bes --dry-run`

1. Grant the pilot group web access **conditioned on** an access level (context-aware binding):

```
gcloud iap web add-iam-policy-binding \
  --project=$PILOT_PROJECT \
  --resource-type=backend-services \
  --service=iap-app-pilot-bes \
  --member=$PILOT_GROUP \
  --role=roles/iap.httpsResourceAccessor \
  --condition="expression=<access-level condition>,title=Corp-Managed-v1"
```

The condition references `Access Level – Corp-Managed – v1`. **Verify** the exact condition expression syntax for access-level conditions in your `gcloud` version; the IAP helper prints the current form for your CLI.

1. Confirm the backend is not reachable except through IAP: the load balancer only accepts IAP-signed

requests, and no firewall rule allows the backend to be hit directly. **Gate 2:** IAP is enabled on `IAP App – Pilot` and the context-aware binding is present.

Stage 3 — Configure IAP TCP forwarding for the pilot VM

Goal: replace bastion/VPN SSH to `IAP VM – Pilot` with brokered, context-checked SSH.

1. Ensure the firewall allows SSH ingress **only from the IAP forwarding range** `35.235.240.0/20`

(requirement 2.3) and remove any rule that allowed SSH from the internet or the broad VPN range. The IAP helper checks this in dry-run: `python bce_iap_enable_helper.py --project $PILOT_PROJECT --resource-type ssh --instance $PILOT_VM --dry-run`

1. Grant the pilot group the TCP-tunnel accessor role on the VM, conditioned on an access level:

```
gcloud iap tcp add-iam-policy-binding \
  --project=$PILOT_PROJECT \
  --member=$PILOT_GROUP \
  --role=roles/iap.tunnelResourceAccessor \
  --condition="expression=<access-level condition>,title=Corp-Managed-v1"
```

(Scope the binding to the instance/zone per your policy; **verify** the exact resource scoping flags for tunnel bindings in your `gcloud` version.)

1. Test the brokered connection: `gcloud compute ssh $PILOT_VM --tunnel-through-iap --zone <zone>`.

Gate 3: SSH to `IAP VM – Pilot` succeeds *only* through `--tunnel-through-iap` and the pilot group, and direct SSH (without IAP) is refused.

Stage 4 — Pilot & Validate (the deny/allow gate)

Goal: prove access is granted on identity+context and denied otherwise, with evidence.

1. Run the validation report tool, which exercises the allow/deny matrix and reads IAP/audit logs:

```
python bce_zerotrust_validation_report.py --project $PILOT_PROJECT --resource iap-app-pilot-bes --group $PILOT_GROUP
```

1. Confirm the four required outcomes (details and expected pass/fail in **05-testing.md**):

• Pilot user on a **corp-managed device from an allowed region** → **allowed**.

• Pilot user on an **unmanaged device / disallowed region** → **denied** (fails the access level).

• **Non-pilot** user (not in the group) → **denied** (fails IAM).

• Direct access **bypassing IAP** → **refused** (backend/VM not reachable directly).

1. Run the guardrail tool to catch weak configuration (unconditioned IAP bindings, access levels with

no device/context condition, SSH firewall not locked to the IAP range): `python bce_policy_guardrail_check.py --project $PILOT_PROJECT --policy $POLICY_ID`.
Gate 4 (go/no-go): all four outcomes are correct **and** the guardrail check reports no high-severity findings. Do not proceed to VPN retirement until this gate passes.

Stage 5 — Retire the VPN path (production-affecting — change window)

Goal: make the app/VM reachable *only* by identity+context, not by network position.

1. In the change window, remove the VPN route and firewall exposure that previously reached the pilot

app/VM (the ones inventoried in Stage 0). The app is now reachable solely through IAP.

1. Re-run `bce_zerotrust_validation_report.py` to confirm access still works through IAP and is now

impossible via the retired path. **Gate 5:** the retired VPN/firewall path no longer reaches the app/VM, and IAP access still passes.

Stage 6 — Expand

Goal: apply the validated pattern to more apps/VMs.

1. Reuse the same access levels across additional resources (define once, reference everywhere).

2. For each new resource repeat Stages 2–5, promoting access-level names to `– v2/– Prod` scopes as

the policy hardens. Version access-level definitions in IaC (Terraform / Cloud Foundation Toolkit) as a **pattern** — representative snippet:

```
# Pattern only — module internals deferred to the provider's docs
resource "google_access_context_manager_access_level" "corp_managed_v1" {
  parent = "accessPolicies/${var.policy_id}"
  name   = "accessPolicies/${var.policy_id}/accessLevels/corp_managed_v1"
  title  = "Access Level – Corp-Managed – v1"
  # basic { conditions { ... device_policy / regions / ip_subnetworks ... } }
}
```

1. Run the scope/config helper's export mode to capture the current zero-trust scope (which resources

are behind IAP, which access levels apply, which broad firewall paths remain) as an inventory for governance sign-off: `python bce_scope_config_helper.py export --org $ORG_ID --project $PILOT_PROJECT --format json`.

Rollback (keep ready before any enforcement change)

• **Access-level binding wrong / users locked out:** remove the access-level condition from the IAP

binding (re-add without `--condition`) or delete the offending binding, restoring identity-only IAM access, then fix the access level and re-bind.

• **IAP web enforcement broke the app:** disable IAP on the backend service (Console: IAP → toggle

off), returning to the prior load-balancer behavior, then diagnose and re-enable.

• **IAP SSH lockout:** re-add a temporary, narrowly-scoped SSH firewall rule from a known admin IP (or

use the serial console) to regain access, then re-lock to `35.235.240.0/20`.

• **VPN retirement caused an outage:** re-apply the removed route/firewall rule from the Stage 0

baseline to restore the prior path, then re-plan the retirement. Only retire again after Gate 4/5 pass cleanly. Every rollback step operates on artifacts named in this guide; log each rollback for the change record.

5. Testing & Validation — BeyondCorp Enterprise: Zero-Trust Access

Each test states the procedure and the **expected pass/fail**. The zero-trust principle is validated only when access is **granted on identity + context** and **denied otherwise** — every test below maps to one of the implementation gates. Run them against the named pilot artifacts (`IAP App – Pilot`, `IAP VM – Pilot`, `Pilot Group – ZeroTrust`, `Access Level – Corp-Managed – v1`) before expanding or retiring any VPN path. The `bce_zerotrust_validation_report.py` tool automates the allow/deny matrix and gathers the log evidence.

T1 — Access levels exist and are well-formed (Gate 1)

• **Procedure:** `gcloud access-context-manager levels list --policy $POLICY_ID`; open each level and

confirm its conditions match the intended identity/device/network design.

• **Pass:** both `Access Level – Trusted-Network – v1` and `Access Level – Corp-Managed – v1` exist,

and the corp-managed level includes a **device** condition (not identity/IP only).

• **Fail:** a level is missing, or the corp-managed level has no device/context condition (the guardrail check flags this) — return to Implementation Stage 1.

T2 — Allowed path: corp-managed device, allowed region (Gate 4)

• **Procedure:** As a **member of** `Pilot Group – ZeroTrust` on a **corporate-managed device** in an **allowed region**, open the pilot app URL and SSH via `gcloud compute ssh $PILOT_VM --tunnel-through-iap`.

• **Pass:** both the app loads and the SSH session connects; IAP logs show an **allow** for that user with the access level satisfied.

• **Fail:** access is denied for a user who should pass — inspect the access-level condition and the device signal (T6); the binding condition may not match the level.

T3 — Denied path: unmanaged device or disallowed region (Gate 4)

• **Procedure:** As the **same pilot user** but from an **unmanaged device** (or a disallowed region / outside the trusted network), retry the app and the SSH tunnel.

• **Pass:** access is **denied** — the request fails the access level even though the identity is authorized; IAP/audit logs show a **deny** attributed to the access-level condition.

• **Fail:** access is granted from an unmanaged/disallowed context — the access level is not being enforced on the binding. Confirm the `--condition` is present on the IAP binding; re-check Stage 2/3.

T4 — Denied path: unauthorized identity (Gate 4)

• **Procedure:** As a user **not** in `Pilot Group – ZeroTrust`, attempt the pilot app and the SSH tunnel from an otherwise-compliant device.

• **Pass:** access is **denied** by IAM (no `roles/iap.*ResourceAccessor` grant); logs show the deny.

• **Fail:** a non-member reaches the resource — an over-broad IAM binding exists (e.g. a grant to `allAuthenticatedUsers` or a wider group). Remove it; the guardrail check flags broad principals.

T5 — Direct-bypass attempt is refused (Gate 2/3/5)

• **Procedure:** Attempt to reach the app backend directly (not through the load balancer/IAP) and SSH to the VM **without** `--tunnel-through-iap`, from both the internet and the (pre-retirement) VPN.

• **Pass:** the direct attempt **fails** — the backend only accepts IAP-signed requests and SSH is only permitted from `35.235.240.0/20`. After Stage 5, the retired VPN path also cannot reach the VM.

• **Fail:** direct or VPN access still works — a firewall rule still exposes the backend/SSH. Lock it down (the guardrail check flags SSH not restricted to the IAP range) and re-test.

T6 — Device-trust signal is actually evaluated

• **Procedure:** Toggle the device out of compliance in the signal source (e.g. remove endpoint enrollment / mark unmanaged) — **verify the exact toggle for your signal source** — and retry.

• **Pass:** access flips from allowed to **denied** when the device signal changes, proving the signal drives the decision (not just IP/identity).

• **Fail:** the decision does not change with device state — the access level is testing only identity/network. Add/repair the device condition (requirement 2.4, Stage 1).

T7 — Guardrail / posture check is clean (Gate 4)

• **Procedure:** `python bce_policy_guardrail_check.py --project $PILOT_PROJECT --policy $POLICY_ID`.

• **Pass:** no high-severity findings — every IAP binding carries an access-level condition, no over-broad principals, access levels include device/context conditions, and SSH firewall is scoped to the IAP range.

• **Fail:** any high-severity finding — remediate before proceeding to VPN retirement (Stage 5).

T8 — VPN retirement verified (Gate 5)

• **Procedure:** After Stage 5, from the retired VPN path attempt the app/VM; then confirm IAP access still works from a compliant device.

• **Pass:** the retired path **cannot** reach the app/VM, **and** IAP access from a compliant pilot user still succeeds.

• **Fail:** the old path still reaches the resource, or IAP access broke — roll back per the Implementation rollback and re-plan.

Evidence to retain

Export the IAP allow/deny logs and the validation-report output for T2–T5 and T8, plus the guardrail report for T7, as the audit evidence that zero-trust enforcement is live for each resource.

6. Troubleshooting — BeyondCorp Enterprise: Zero-Trust Access

Common failure modes and resolutions. Symptoms reference the named artifacts (`IAP App – Pilot`, `IAP VM – Pilot`, `Pilot Group – ZeroTrust`, `Access Level – Corp-Managed – v1`) and the

gates from Implementation and Testing. Where a fix depends on a version- or preview-dependent surface, it is flagged **verify** — confirm the current path in your organization.

Access levels won't create / "no access policy"

- **Symptom:** `gcloud access-context-manager levels create ...` fails, or `policies list` returns

nothing.

- **Cause:** The Access Context Manager API is not enabled, or no organization-scoped access policy

exists (requirement 2.1).

- **Resolution:** Enable the API (`gcloud services enable accesscontextmanager.googleapis.com`) and

confirm/create the org access policy. Only **one scoped access policy** applies to the whole organization — if one already exists (often from the Landing Zone or VPC-SC solution), reuse its policy ID rather than creating a second.

IAP page won't enable for the web app / OAuth consent error

- **Symptom:** Turning on IAP for the backend service errors, or users hit an OAuth consent problem.
- **Cause:** `iap.googleapis.com` not enabled, or the project's OAuth brand/consent screen isn't

configured (required the first time IAP for web is used).

- **Resolution:** `gcloud services enable iap.googleapis.com`; configure the OAuth brand/consent as the

console prompts. This first-run flow is **version-dependent** — follow the in-console prompt / current IAP docs. Re-run `bce_iap_enable_helper.py --resource-type web --dry-run` to re-check prerequisites.

Access is granted from an unmanaged device or wrong region (T3 fails)

- **Symptom:** A pilot user reaches the app/VM even from a non-compliant context.
- **Cause:** The IAP binding has **no access-level condition** — it enforces identity (IAM) but not

context — or the condition references the wrong access level.

- **Resolution:** Confirm the binding carries a `--condition` referencing `Access Level – Corp-Managed –

v1. Re-add the binding with the condition (Implementation Stage 2/3). bce_policy_guardrail_check.py flags this as IAP_NO_CONDITION (HIGH). **Verify** the exact condition-expression syntax for your gcloud` version.

Device state changes but the decision doesn't (T6 fails)

- **Symptom:** Marking a device unmanaged/non-compliant doesn't flip access from allow to deny.

• **Cause:** The access level tests only identity/network, not a device signal; or the device-trust signal source isn't enrolled/licensed for the pilot devices (requirement 2.4).

• **Resolution:** Add a **device condition** to the access level and confirm the signal source is live and licensed. Richer device signals may require **Chrome Enterprise Premium / endpoint-verification enrollment — verify the exact signal source and license in your organization.** Rebuild the level with `bce_scope_config_helper.py build --require-corp-device`.

A user who should have access is denied (T2 fails)

• **Symptom:** A compliant member of `Pilot Group – ZeroTrust` is denied.

• **Cause:** The user isn't in the group; the group lacks `roles/iap.httpsResourceAccessor` / `roles/iap.tunnelResourceAccessor`; the access-level condition is stricter than intended (e.g. region or IP mismatch); or the device signal isn't reporting for that device.

• **Resolution:** Verify group membership and the IAM binding; inspect the access-level condition against the user's actual context (IP/region/device). Loosen or correct the level, then re-test. IAP/audit logs show which condition produced the deny.

SSH via `--tunnel-through-iap` fails

• **Symptom:** `gcloud compute ssh <vm> --tunnel-through-iap` times out or is refused.

• **Cause:** The firewall doesn't allow SSH from the IAP range; the user lacks `roles/iap.tunnelResourceAccessor` (optionally conditioned); or `iap.googleapis.com` isn't enabled.

• **Resolution:** Ensure an ingress rule allows `tcp:22` **from** `35.235.240.0/20` (and not from the internet/VPN); grant the tunnel accessor role to `Pilot Group – ZeroTrust`; enable the IAP API. `bce_iap_enable_helper.py --resource-type ssh --dry-run` prints and checks all three.

Direct access to the backend/VM still works (T5 fails)

• **Symptom:** The app backend is reachable without IAP, or SSH works without the IAP tunnel.

• **Cause:** A firewall rule still exposes the backend/SSH directly (often a broad `0.0.0.0/0` rule or a leftover VPN-range rule).

• **Resolution:** Remove the broad rule; scope SSH to `35.235.240.0/20` and the web backend to accept only IAP-signed traffic. `bce_policy_guardrail_check.py` flags `SSH_BROAD` (HIGH); `bce_access_assessment.py` tags such rules `FLAT-TRUST`.

An unauthorized user reaches the resource (T4 fails)

• **Symptom:** Someone not in `Pilot Group – ZeroTrust` gets in.

• **Cause:** An over-broad IAM binding — e.g. a grant to `allAuthenticatedUsers`/`allUsers` or a wider

group.

• **Resolution:** Remove the broad binding; bind only the pilot group. The guardrail check flags

`IAP_BROAD_PRINCIPAL` (HIGH).

VPN retirement locked users out (Stage 5)

• **Symptom:** After removing a VPN route/firewall rule, users can no longer reach the app/VM.

• **Cause:** The app/VM wasn't fully reachable through IAP before the path was retired (Gate 4 not truly

passed), or an internal dependency still relied on the retired path.

• **Resolution: Roll back** — re-apply the removed route/firewall rule from the Stage 0 baseline to

restore access. Confirm IAP access works for a compliant pilot user (Gate 4) *before* retiring again. Retire per-resource, not in bulk.

`gcloud` access-level / IAP flags differ from the guide

• **Symptom:** A flag (`--basic-level-spec`, `--custom-level-spec`, condition syntax, tunnel binding

scoping) is rejected or behaves differently.

• **Cause:** These surfaces are **version-dependent** and evolve.

• **Resolution:** Run `gcloud <command> --help` for your installed version and confirm the current flag

shape; the guide flags these as **verify**. Update `gcloud` if it is significantly out of date.

Guardrail check can't read policies/logs

• **Symptom:** `bce_policy_guardrail_check.py` or the validation report prints `*_UNREADABLE` / no log

entries.

• **Cause:** The running identity lacks read roles (`roles/accesscontextmanager.policyReader`,

IAP policy read, `roles/logging.viewer`) or Cloud Audit Logging for IAP isn't enabled.

• **Resolution:** Grant the read roles to the operator/service identity via ADC; enable Data Access

audit logs for IAP so allow/deny evidence is captured. The tools are read-only and safe to re-run.

7. Best Practices — BeyondCorp Enterprise: Zero-Trust Access

Durable practices for running and expanding BeyondCorp Enterprise zero-trust access after the pilot.

Policy design

- **Define access levels once, reference everywhere.** Keep a small, well-named set (e.g.

`Access Level – Corp-Managed – v1`, `Access Level – Trusted-Network – v1`) rather than a bespoke condition per app. Fewer, reusable levels are easier to reason about and audit.

- **Always condition IAP bindings on an access level.** An IAP binding without a `--condition` enforces

identity but not context — that is not zero trust. Treat an unconditioned binding as a defect (the guardrail check flags it).

- **Prefer device + identity + network together** over any single attribute. Network (IP/geo) alone is

the flat-trust model you are replacing; identity alone ignores the "what."

- **Use CEL custom access levels for combined logic**, and keep the CEL expressions in version control

so changes are reviewable and revertible.

Rollout discipline

- **Pilot-first, gate-driven.** Never enable enforcement or retire a VPN path without passing the

deny/allow gate (05-testing T2–T5) and the guardrail check (T7). Keep the documented rollback ready before every enforcement change.

- **Retire VPN paths per resource, not all at once.** Move an app/VM behind IAP, validate, then remove

only that path. This keeps blast radius small and rollback simple.

- **Version access-level names as policy hardens** (`– v1` → `– v2`, `– Pilot` → `– Prod`) and update

bindings deliberately rather than mutating a live level in place.

Keep the two products distinct

- **BeyondCorp Enterprise (this solution) protects your apps/VMs/GCP**, in the **Google Cloud

Console**; Workspace Context-Aware Access protects Google Workspace apps**, in the** Google Admin console**. Document which surface owns which control so they are not conflated, even though they can share device signals.

Device signals & licensing

- **Confirm the device-signal source and its license tier before relying on it.** Richer device-trust

signals may require Chrome Enterprise Premium / endpoint-verification enrollment — **verify in your organization** and re-verify at renewal, since tiers and signals change.

- **Degrade gracefully:** if a device signal is temporarily unavailable, know whether your access level

fails open or closed and make that an explicit, tested decision (default to fail-closed for sensitive apps).

Operations, monitoring & evidence

- **Treat IAP and Cloud Audit Logs as the source of truth** for who reached what, from what context.

Route allow/deny events to your SIEM and alert on unexpected denies (possible mis-scoped level) and unexpected allows (possible over-broad binding).

- **Re-run the guardrail check on a schedule**, not just at rollout, to catch drift (a new unconditioned

binding, an SSH firewall opened for troubleshooting and left open, a broad principal added).

- **Review access-level membership and bindings periodically** with the security stakeholders; remove

access that is no longer needed (least privilege per app).

Least privilege for administrators

- Grant `roles/accesscontextmanager.policyAdmin` and `roles/iap.admin` narrowly to the people who

configure zero trust; keep org-super-admin as audited break-glass only.

- Automate with **Application Default Credentials / short-lived tokens** — never export service-account

keys to disk or embed secrets in scripts (the provided tools follow this).

Governance & scope

- **Maintain a living inventory** of what is behind IAP, which access levels apply, and which VPN paths

remain (export it with `bce_scope_config_helper.py`). Zero-trust coverage is a program, not a one-time project — track expansion against the remaining VPN-dependent apps.

- **Defer legal/records/compliance determinations** to those functions; provide the technical control

and the log evidence, not the legal interpretation.

8. Deliverables Summary — BeyondCorp Enterprise: Zero-Trust Access

Documentation

- **01 Solution Overview** — the zero-trust model (Access Context Manager access levels, Identity-Aware

Proxy, context-aware access bindings, device-trust signals), the reference design, benefits/limits, the deployment model, and how this differs from Workspace Context-Aware Access.

- **02 Requirements** — solution-specific prerequisites (access policy, pilot app behind an HTTPS load

balancer, pilot VM for IAP TCP forwarding, device-signal source + license, pilot group + admin roles, IAP/API enablement, rollback/change window), building on the shared GCP global requirements.

- **03 Implementation** — ordered Assess → Design → Configure → Pilot → Validate → Expand steps: define

access levels, front a pilot app and VM with IAP, add context-aware bindings and device signals, validate deny/allow, retire the VPN path, and expand — with pass/fail gates and a rollback.

- **05 Testing & Validation** — the deny/allow matrix (allowed on identity+context, denied on

unmanaged/wrong-region, denied for unauthorized identity, direct-bypass refused), device-signal and guardrail checks, and VPN-retirement verification, each with expected pass/fail.

- **06 Troubleshooting** — common failure modes (IAP not enforcing, access-level condition not applied,

device signal not evaluated, SSH-over-IAP failures, VPN-retirement lockouts) and resolutions.

- **07 Best Practices** — durable practices for reusable access levels, gate-driven rollout, keeping the

two products distinct, device-signal/licensing hygiene, monitoring, least-privilege admin, and scope governance.

- **Executive Summary** — the outcome, approach, and why it is safe and governed, for decision-makers.

The 5 Python tools (in `scripts/`)

Automation is Python over the `gcloud` CLI / Google Cloud Client Libraries. Every tool is **safe-by-default** (read-only or `--dry-run` unless `--apply` is passed), uses **Application Default Credentials / the environment** for auth, and contains **no hardcoded credentials, service-account keys, or secrets**.

1. `bce_access_assessment.py` — *Assessment / discovery (read-only).* Reports the current access

policies and access levels, which candidate resources already have IAP enabled, and firewall rules that broadly expose SSH or HTTP(S) (a flat-trust / VPN-dependency signal). Produces the Stage 0 baseline. Read-only.

1. `bce_iap_enable_helper.py` — *IAP enablement helper for an app or VM (dry-run default).* Checks

prerequisites and prints the exact steps to enable IAP for a web backend service or IAP TCP forwarding for a VM (including the `35.235.240.0/20` firewall requirement and the context-aware binding form). Applies only with `--apply`.

1. `bce_zerotrust_validation_report.py` — *Validation / report generator (read-only).* Exercises the

allow/deny matrix for a resource and pilot group and reads IAP/audit-log evidence, producing the deny/allow validation report that backs the Stage 4 and Stage 5 gates.

1. `bce_policy_guardrail_check.py` — *Policy / guardrail check (read-only).* Flags weak

configuration: IAP bindings without an access-level condition, over-broad principals, access levels lacking a device/context condition, and SSH firewall rules not scoped to the IAP range. High-severity findings block VPN retirement.

1. `bce_scope_config_helper.py` — *Scope / config helper (dry-run default for writes).* Two modes:

(a) **build** an Access Context Manager access level from basic attribute conditions or a CEL custom-level skeleton, printing the exact `gcloud`/YAML it would apply (no change unless `--apply`); and (b) **export** the current zero-trust scope — which resources are behind IAP, which access levels apply, which broad firewall paths remain — as a JSON/table inventory for governance sign-off (read-only).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

bce_access_assessment.py

bce_iap_enable_helper.py

bce_policy_guardrail_check.py

bce_scope_config_helper.py

bce_zerotrust_validation_report.py